

Documentacion Tecnica delCodigo

Sistema Inteligente de Reconocimiento Facial
y Analisis de Probabilidades

Como funciona todo: Deep Learning, Machine Learning, flujo, estructura del
codigo, API, base de datos y despliegue.

Grupo 04 - SENATI
IA, Machine Learning y Deep Learning

1. Resumen del sistema

Es una aplicacion web que **registra personas, reconoce rostros y estima la probabilidad** de que una coincidencia sea correcta. Combina tres campos: Inteligencia Artificial (el todo), Machine Learning (aprende de datos) y Deep Learning (redes neuronales profundas para los rostros).

El sistema entrega uno de **tres resultados** al analizar una imagen:

- **Humano Real:** rostro que coincide con una persona registrada.
- **No Real:** hay rostro pero no coincide con nadie registrado (desconocido).
- **No es humano:** no se detecto un rostro (objeto, animal o dibujo).

2. Arquitectura y stack tecnologico

Capa	Tecnologia	Responsabilidad
Frontend	React + TypeScript + Vite + Tailwind	Interfaz, camara, formularios y graficos
Backend	Python + FastAPI + Uvicorn	API REST, logica y seguridad
Deep Learning	InsightFace (ArcFace, buffalo_s)	Deteccion de rostro y embeddings
Machine Learning	scikit-learn (Regresion Logistica)	Clasificacion y probabilidad calibrada
Vision	OpenCV + NumPy	Calidad de imagen e iluminacion
Base de datos	PostgreSQL + pgvector (Supabase)	Personas, embeddings, logs, ML
Despliegue	Vercel + Railway + Supabase	Frontend, backend y datos en la nube

Flujo general (una imagen entra y se procesa asi):

```
IMAGEN
|
[DL] Detector: hay un rostro humano? --NO--> "No es humano"
|SI
[DL] ArcFace: genera embedding (512 numeros)
|
[DL] pgvector: busca el mas parecido (similitud coseno)
|
[ML] scikit-learn: con 4 variables -> probabilidad calibrada
|
RESULTADO: Humano Real / No Real
```

3. Deep Learning (DL): reconocer rostros

El Deep Learning se encarga de **ver y reconocer**. Tiene dos trabajos:

3.1. Deteccion y embedding

El modelo **InsightFace (buffalo_s / ArcFace)** detecta el rostro en la imagen y lo convierte en un **embedding**: un vector de 512 numeros que representa de forma unica esa cara. Si no encuentra un rostro humano (objeto, animal, dibujo), el sistema responde **"No es humano"**.

Archivo: backend/app/services/face_service.py

```
faces = self.app.get(img)          # detecta rostros
if not faces:
```

```
raise ValueError('No se detecto ningun rostro') # -> No es humano
embedding = faces[0].normed_embedding # vector de 512 (ArcFace)
```

3.2. Comparacion (busqueda 1:N)

El embedding se compara contra los guardados con **similitud coseno** usando **pgvector** (indice HNSW). Mientras mas cercano a 1, mas parecido. La **distancia** es lo inverso (1 - similitud).

Archivo: backend/app/ml/vector_store.py

4. Machine Learning (ML): clasificar y dar probabilidad

El ML no tiene reglas escritas a mano: **aprende** de los reconocimientos que tu **confirmas**. Con eso clasifica (Humano Real / No Real) y calcula la **probabilidad calibrada**.

4.1. Las 4 variables de entrada (segun el PDF del proyecto)

Variable	Que mide	Rango
Similitud	Que tan parecido es al rostro registrado	0 a 1
Distancia	Lo inverso de la similitud (1 - similitud)	0 a 2
Calidad de imagen	Nitidez del rostro (varianza Laplaciana)	0 a 1
Iluminacion	Brillo promedio del rostro	0 a 1

La variable objetivo (lo que aprende a predecir) es **resultado_real** (Humano Real / No Real), que sale de tus confirmaciones. **Ojo:** el umbral y la confianza NO son entradas del modelo (el umbral es una regla fija; la confianza es un resultado que sale de la probabilidad).

4.2. Entrenamiento y metricas

- Modelo: **Regresion Logistica** (class_weight=balanced) + calibracion (isotonica/sigmoid).
- Minimo **10 ejemplos de cada clase** (10 Humano Real + 10 No Real) para entrenar.
- Con datos suficientes hace un **holdout 75/25**: entrena con 75% y evalua con 25% no visto.
- Metricas (Seccion 7 del PDF): accuracy, precision, recall, F1, FAR, FRR y matriz de confusion.

Archivos: backend/app/ml/trainer.py y backend/app/services/probability_service.py

5. Diferencia clave: DL vs ML

	Deep Learning (DL)	Machine Learning (ML)
Trabajo	Ver y reconocer el rostro	Aprender y dar la probabilidad
Responde	Que es y quien es	Que tan confiable es
Herramienta	InsightFace / ArcFace	scikit-learn
Necesita entrenar?	No (modelo preentrenado)	Si (con tus confirmaciones)

6. Flujo completo de uso

- **1. Registrar:** se ingresa nombre, correo, consentimiento y 1 foto del rostro. El DL genera y guarda el embedding.
- **2. Reconocer:** por camara o subida. El sistema detecta, compara y da el resultado.
- **3. Confirmar:** el administrador marca cada reconocimiento como Humano Real o No Real. Esto crea los datos de entrenamiento.
- **4. Entrenar:** con 10+10 confirmados, el ML se entrena y muestra las metricas.
- **5. Analizar:** en el Laboratorio de Pruebas (1:N, 1:1, Manual) se ven las variables y la probabilidad.

7. Los tres resultados posibles

Resultado	Cuando ocurre	Acceso
Humano Real	Hay rostro y coincide con un registrado	Permite
No Real	Hay rostro pero no coincide (desconocido)	No permite
No es humano	No se detecto rostro (objeto/animal/dibujo)	No permite

La deteccion de "No es humano" la hace el **detector del DL** (si no hay rostro), no el ML. Es confiable para distinguir personas de objetos, animales y dibujos.

8. Estructura del codigo

8.1. Backend (FastAPI)

```
backend/app/  
  main.py          -> arranque de la API  
  core/config.py   -> configuracion (umbral, MIN_TRAIN_PER_CLASS...)  
  core/supabase_client.py -> conexion a Supabase  
  services/  
    face_service.py   -> DL: deteccion + embedding + calidad/luz  
    embedding_service.py -> guardar/leer embeddings  
    probability_service.py -> ML: predecir probabilidad  
    audit_service.py  -> logs y datos de entrenamiento  
  ml/  
    trainer.py        -> ML: entrenar + metricas (holdout)  
    vector_store.py   -> busqueda 1:N con pgvector  
  api/routes/  
    personas / recognition / probabilities / models / auth / dashboard
```

8.2. Frontend (React)

```
frontend/src/  
  pages/          -> Dashboard, RegistroFacial, Reconocimiento,  
                   Probabilidades, Historial, EntrenamientoML, Seguridad  
  components/     -> CameraCapture, FaceResultCard, ChatBot, etc.  
  services/api.ts -> llamadas a la API del backend
```

9. API REST (endpoints principales)

Metodo y ruta	Funcion
POST /api/personas	Registrar persona (con rostro)
POST /api/reconocimiento	Reconocer un rostro

Metodo y ruta	Funcion
GET /api/reconocimiento/historial	Historial de reconocimientos
PATCH /api/reconocimiento/historial/{id}/confirmar	Confirmar Humano Real / No Real
POST /api/probabilidades/comparar-1n	Comparar 1 contra todas
POST /api/probabilidades/comparar-1a1	Comparar dos personas
POST /api/probabilidades/predecir-manual	Prediccion con sliders
POST /api/modelos/entrenar	Entrenar el modelo ML
GET /api/modelos/metricas	Ver metricas del modelo

10. Base de datos (Supabase / PostgreSQL)

Tabla	Campos principales
personas	id, nombre, email, activo, created_at
face_embeddings	id, persona_id, embedding (vector 512), modelo
recognition_logs	id, persona_id, similitud, distancia, umbral, coincide, probabilidad_calibrada, calidad_imagen, iluminacion
ml_training_records	id, similitud, distancia, calidad_imagen, iluminacion, resultado_real
users	id, name, email, role (admin/operador/analista)

Los embeddings se guardan como vectores de 512 dimensiones y se buscan con un indice **HNSW** de pgvector. Se aplican politicas RLS (Row Level Security).

11. Modulos (pantallas del frontend)

Modulo	Que hace
Dashboard	Resumen: personas, reconocimientos, estado ML/DL, graficos
Registro Facial	Registrar personas y confirmar reconocimientos
Reconocimiento	Identificar en vivo (Humano Real / No Real / No es humano)
Probabilidades	Laboratorio: 1:N, 1:1 y Manual con las 4 variables
Historial	Lista de reconocimientos y su estado
Entrenamiento ML	Entrenar el modelo y ver metricas y matriz de confusion
Seguridad	Usuarios, roles, permisos y auditoria

12. Despliegue

- **GitHub**: el codigo vive en el repositorio. Al hacer push, se disparan los despliegues.
- **Vercel**: despliega el frontend automaticamente.
- **Railway**: despliega el backend (Docker) automaticamente.
- **Supabase**: base de datos PostgreSQL + pgvector + almacenamiento.

Las credenciales secretas (service_role, contrasena de BD, JWT) van solo en las variables de entorno de Railway, nunca en el repositorio.

13. Como ejecutar localmente

```
# Backend
cd backend
python -m venv venv && venv\Scripts\activate
pip install -r requirements.txt
uvicorn app.main:app --reload --port 8000
```

```
# Frontend
cd frontend
npm install
npm run dev
```

14. Seguridad y proteccion de datos (Ley N. 29733)

- Consentimiento informado obligatorio antes de registrar datos biometricos.
- Se guarda el embedding (vector), no necesariamente la foto original.
- Acceso por roles: administrador, operador y analista.
- HTTPS, autenticacion por token y auditoria de operaciones.

Documento generado automaticamente a partir del codigo del proyecto.